

计算机操作系统

倪福川

fcni_cn@mail.hzau.edu.cn

华中农业大学 信息学院

第四章 存储器管理

4.1 存储器的层次结构

4.2 程序的装入和链接

4.3 连续分配存储管理方式

4.4 对换(Swapping)

4.5 分页存储管理方式

4.6 分段存储管理方式

4.5 分页存储管理方式

- (1) 分页存储管理方式。
- (2) 分段存储管理方式。
- (3) 段页式存储管理方式。

4.5.1 分页存储管理的基本方法

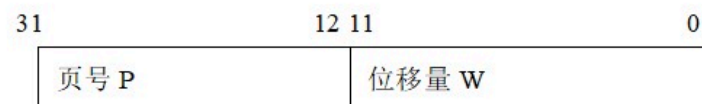
1. 页面和物理块

(1) 页面。

(2) 页面大小。

2. 地址结构

分页地址中的地址结构:



对某特定机器，其地址结构是一定的。若给定一个逻辑地址空间中的地址为A，页面的大小为L，则页号P和页内地址d可按下式求得：

$$P = \text{INT}\left[\frac{A}{L}\right], \quad d = [A] \text{ MOD } L$$

3.页表

将进程的各个页离散地存储在内存的任一物理块中， 为保证进程仍然能够正确地运行， 即能在内存中找到每个页面所对应的物理块， 系统又为每个进程建立了一张**页面映像表**， 简称**页表**。

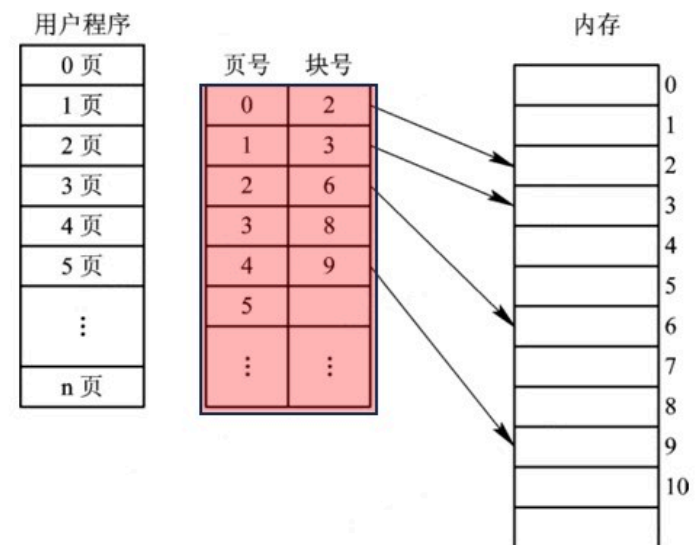


图4-14 页表的作用

4.5.2 地址变换机构

1. 基本的地址变换机构

将用户地址空间中的逻辑地址变换为内存空间中的物理地址，

执行频率非常高，每条指令的地址都需要进行变换，因此需要采用**硬件**来实现。

页表功能是由一组专门的**寄存器**来实现的。一个页表项用一个寄存器。

分页系统的地址变换

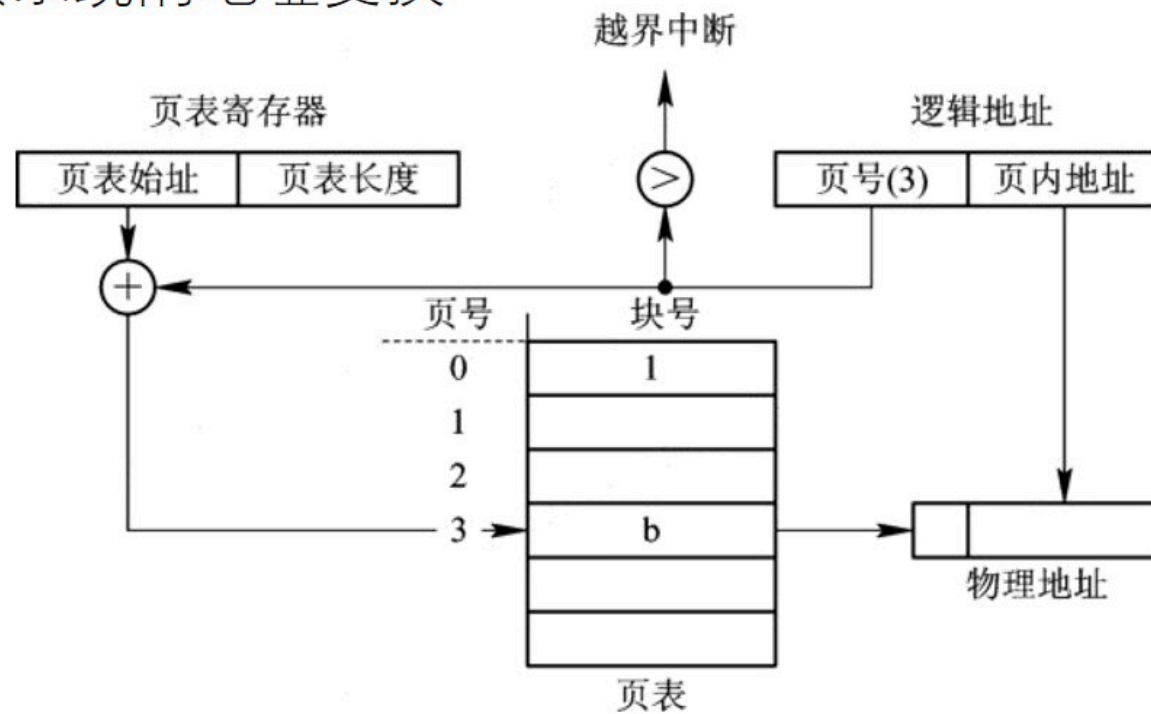


图4-15 分页系统的地址变换机构

2. 具有快表的地址变换机构

页表存放在内存中，CPU在每存取一个数据时，都要**两次访问内存**。

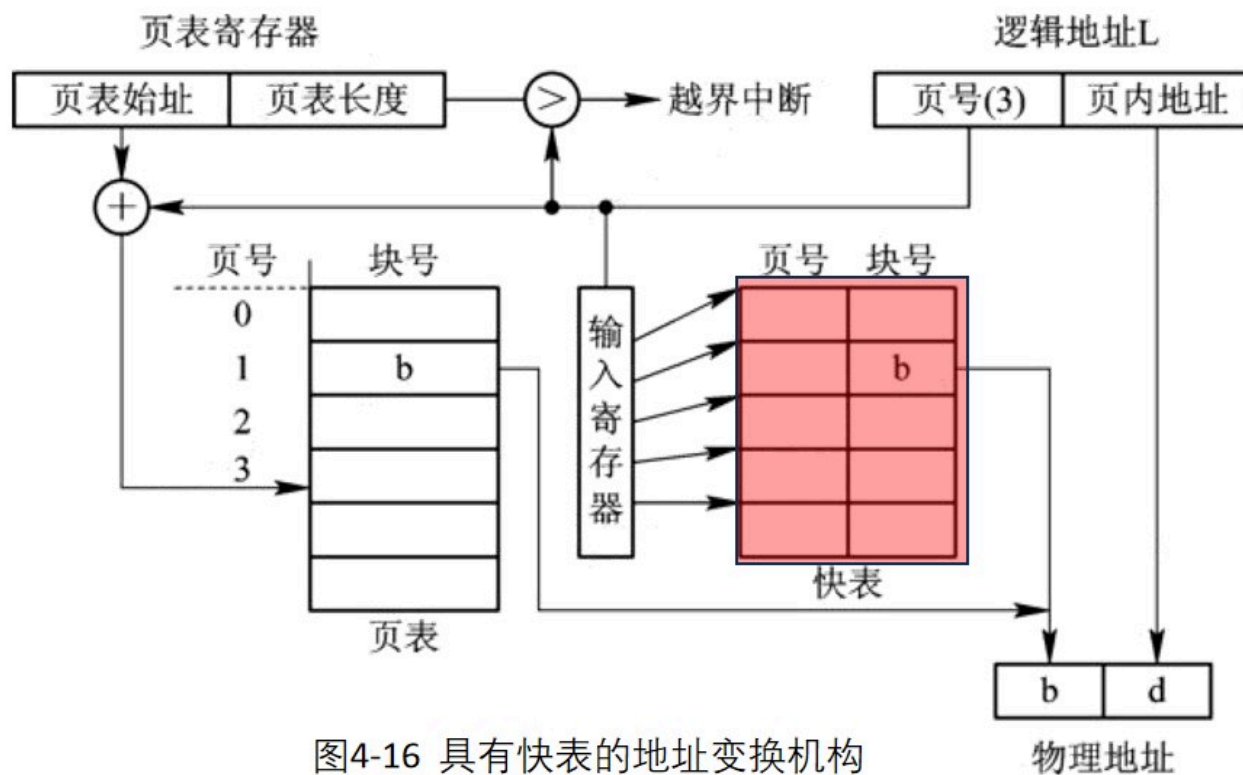
第一次访问内存中页表，找到指定页的物理块号，将**块号**与**页内偏移量W**拼接，以**形成物理地址**。第二次访问内存时，才是从第一次所得地址中**获得所需数据**(或向此地址中写入数据)。

得不偿失!

使计算机的处理速度降低近 $1/2$ ，以高昂代价来换取存储器空间利用率的提高。

块表 /TLB

具有快表的地址变换



4.5.3 访问内存的有效时间

从进程发出指定逻辑地址的访问请求，经过地址变换，到
在内存中找到对应的实际物理地址单元并取出数据，所需要花费
的总时间，称为**内存的有效访问时间**(Effective Access Time, EAT)。

$$EAT = t + t = 2t$$

访问一次内存的时间为t

在引入快表的分页存储管理方式中，通过快表查询，可以直接得到**逻辑页**所对应的**物理块号**，由此拼接形成实际物理地址，减少了一次内存访问，缩短了进程访问内存的有效时间。

$$EAT = a \times \lambda + (t + \lambda)(1 - a) + t = 2t + \lambda - t \times a$$

λ 表示查找快表所需要的时间， a 表示**命中率**， t 表示访问一次内存所需要的时间。

假设对快表的访问时间 λ 为20 ns(纳秒)，对内存的访问时间 t 为100 ns，则下表中列出了不同的命中率 a 与有效访问时间的关系

命中率 (%)	有效访问时间
a	EAT
0	220
50	170
80	140
90	130
98	122

4.5.4 两级和多级页表

1. 两级页表(Two-Level Page Table)

针对难于找到大的连续的内存空间来存放页表的问题，可利用将页表进行分页的方法，使每个页面的大小与内存物理块的大小相同，并为它们进行编号，即依次为0# 页、1# 页，...，n# 页，然后离散地将各个页面分别存放在不同的物理块中。

同样，也要为离散分配的页表再建立一张页表，称为外层页表(Outer Page Table)，在每个页表项中记录了页表页面的物理块号。

两级页表的地址变换

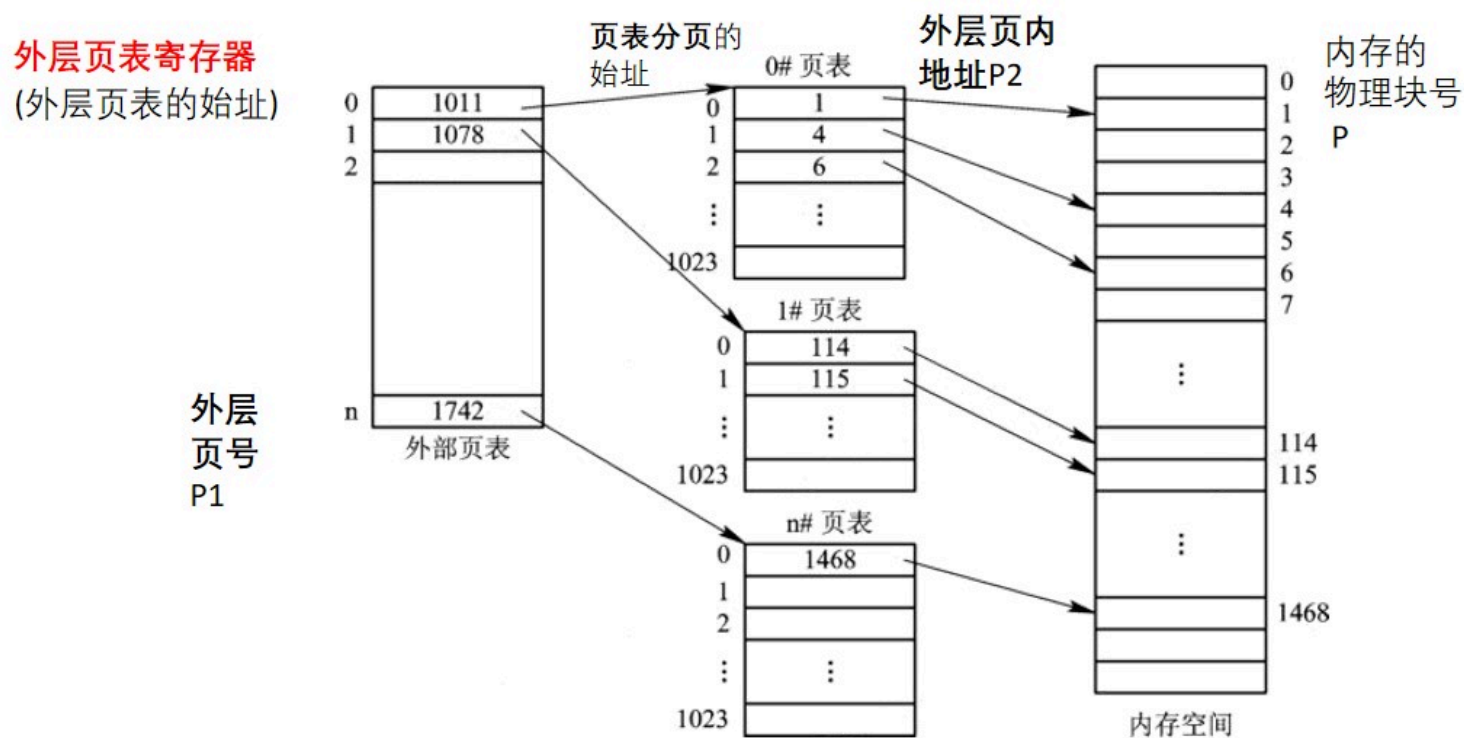


图4-18 具有两级页表的地址变换机构

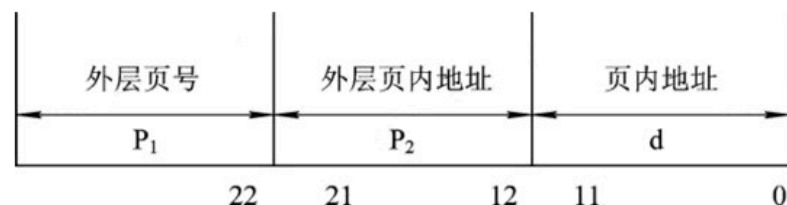


图4-17 两级页表结构

为了方便实现地址变换，在地址变换机构中，同样需要增设一个**外层页表寄存器**，用于存放外层页表的始址，并利用逻辑地址中的外层页号作为外层页表的索引，从中找到指定页表分页的始址，再利用P2作为指定页表分页的索引，找到指定的页表项，其中即含有该页在内存的物理块号，用该块号P和页内地址d即可构成访问的内存物理地址。图4-18示出了两级页表时的地址变换机构。

2. 多级页表

对于32位的机器，采用两级页表结构合适。

64位的机器，采用两级页表是否仍然合适？

64位的机器，如果页面大小仍采用4 KB即 2^{12} B，那么还剩下52位，假定仍按物理块的大小(2^{12} 位)来划分页表，则将余下的42位用于外层页号。

此时在外层页表中可能有4096 G个页表项，要占用16 384 GB的连续内存空间。

4.5.5 反置页表(Inverted Page Table)

1. 反置页表的引入

在分页系统中，为每个进程配置了一张页表，进程逻辑地址空间中的每一页，在页表中都对应有一个页表项。

现代计算机系统中进程的逻辑地址空间非常大，需要有许多页表项，占用大量的内存空间。

2. 地址变换

在利用反置页表进行地址变换时，是根据**进程标识符**和**页号**，去检索反置页表。如果检索到与之匹配的页表项，则该页表项(中)的序号*i*便是该页所在的**物理块号**，可用该块号与页内地址一起构成物理地址送内存地址寄存器。

若检索了整个反置页表仍未找到匹配的页表项，则表明此页尚未装入内存。对于不具有请求调页功能的存储器管理系统，此时则表示地址出错。

对于具有请求调页功能的存储器管理系统，此时应产生请求调页中断，系统将把此页调入内存。

4.6 分段存储管理方式

存储管理方式随着OS的发展也在不断地发展。当OS由单道向多道发展时，存储管理方式便由单一连续分配发展为固定分区分配。

4.6.1 分段存储管理方式的引入

1. 方便编程

通常，用户把自己的作业按照逻辑关系划分为若干个段，每个段都从0开始编址，并有自己的名字和长度。因此，程序员们都迫切地需要访问的逻辑地址是由段名(段号)和段内偏移量(段内地址)决定的，这不仅可以方便程序员编程，也可使程序非常直观，更具可读性。

例如，下述的两条指令便使用段名和段内地址：

```
LOAD 1, [A] | <D> ;  
STORE 1, [B] | <C> ;
```

2. 信息共享

在实现对程序和数据的共享时，是以信息的逻辑单位为基础的。

比如，为了共享某个过程、函数或文件。

分页系统中的“页”只是存放信息的物理单位(块)，并无完整的逻辑意义，这样，一个可被共享的过程往往可能需要占用数十个页面，这为实现共享增加了困难。

3. 信息保护

信息保护同样是以信息的逻辑单位为基础的，而且经常是以一个过程、函数或文件为基本单位进行保护的。

4. 动态增长

在实际应用中，往往存在着一些段，尤其是数据段，在它们的使用过程中，由于数据量的不断增加，而使数据段动态增长，相应地它所需要的存储空间也会动态增加。

然而，对于数据段究竟会增长到多大，事先又很难确切地知道。对此，很难采取预先多分配的方法进行解决。

5. 动态链接

运行时动态链接: 为了提高内存的利用率，系统只将真正要运行的目标程序装入内存，也就是说，动态链接在作业运行之前，并不是把所有的目标程序段都链接起来。

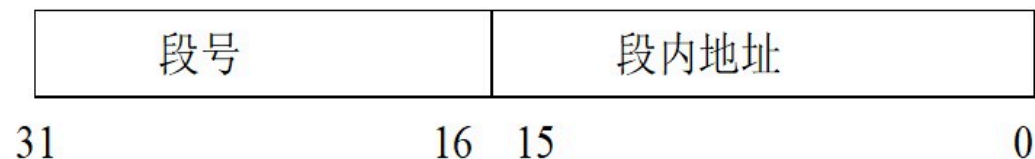
当程序要运行时，首先将主程序和它立即需要用到的目标程序装入内存，即启动运行。而在程序运行过程中，当需要调用某个目标程序时，才将该段(目标程序)调入内存并进行链接。

4.6.2 分段系统的基本原理

1. 分段

在分段存储管理方式中，作业的地址空间被划分为若干个段，每个段定义了一组逻辑信息。例如，有主程序段MAIN、子程序段X、数据段D及栈段S等。

分段地址中的地址结构：



2. 段表

动态分区分配方式中，系统为整个进程分配一个连续的内存空间。

分段式存储管理系统中，则是为**每个分段分配一个连续的分区**。

进程中的各个段，可以**离散**地装入内存中不同的分区中。

为保证程序能正常运行，就必须能从物理内存中找出每个逻辑段所对应的位置。

段表实现地址映射

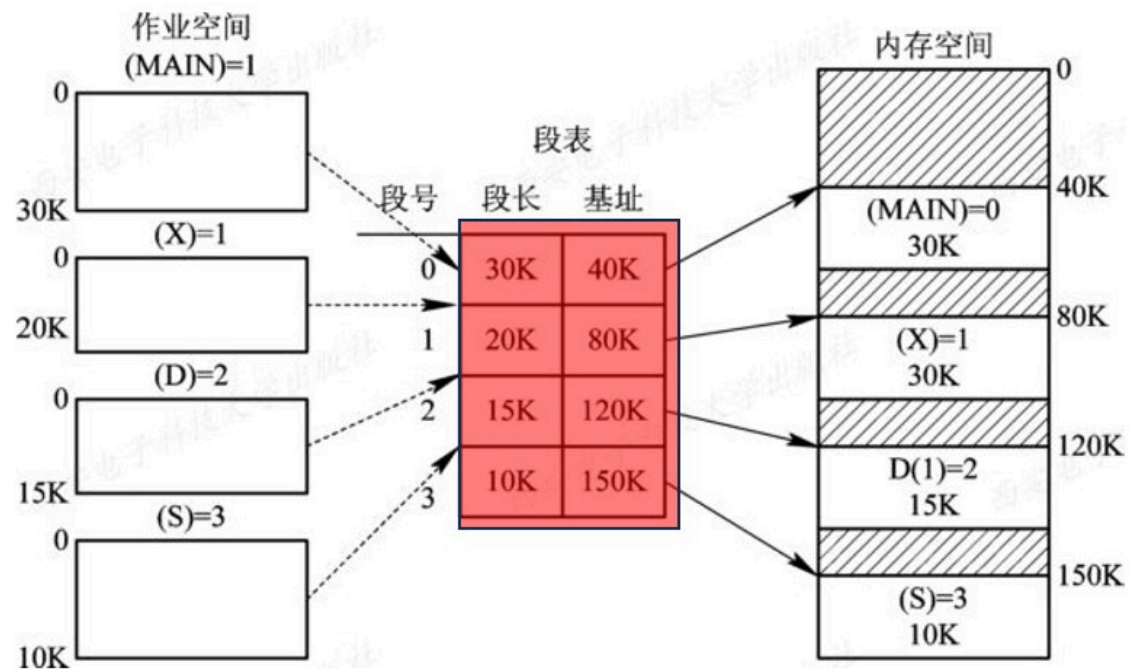


图4-19 利用段表实现地址映射

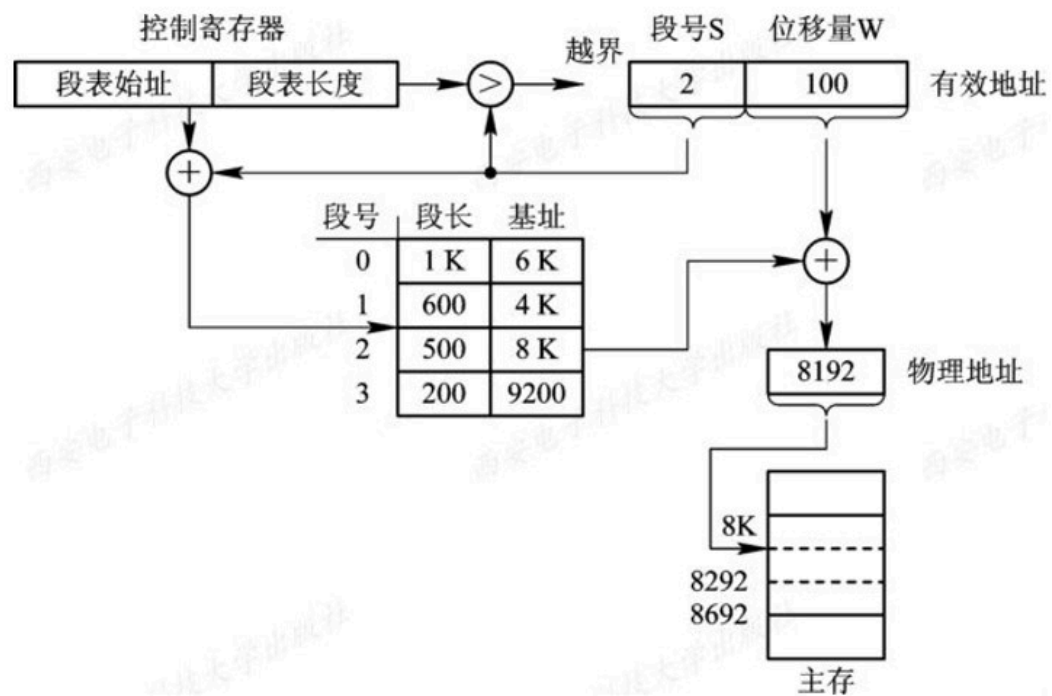
3. 地址变换机构

为了实现进程从逻辑地址到物理地址的变换功能，在系统中设置了**段表寄存器**，用于存放段表始址和段表长度TL。

在进行地址变换时，系统将逻辑地址中的**段号与段表长度TL进行比较**。若 $S > TL$ ，表示段号太大，是访问越界，于是产生越界中断信号。若未越界，则根据段表的始址和该段的段号，计算出该段对应段表项的位置，从中读出该段在内存的起始地址。

然后，再检查**段内地址d是否超过该段的段长SL**。若超过，即 $d > SL$ ，同样发出越界中断信号。若未越界，则将该段的基址d与段内地址相加，即可得到要访问的内存物理地址。图4-20示出了分段系统的地址变换过程。

分段系统的地址变换



• 图4-20 分段系统的地址变换过程

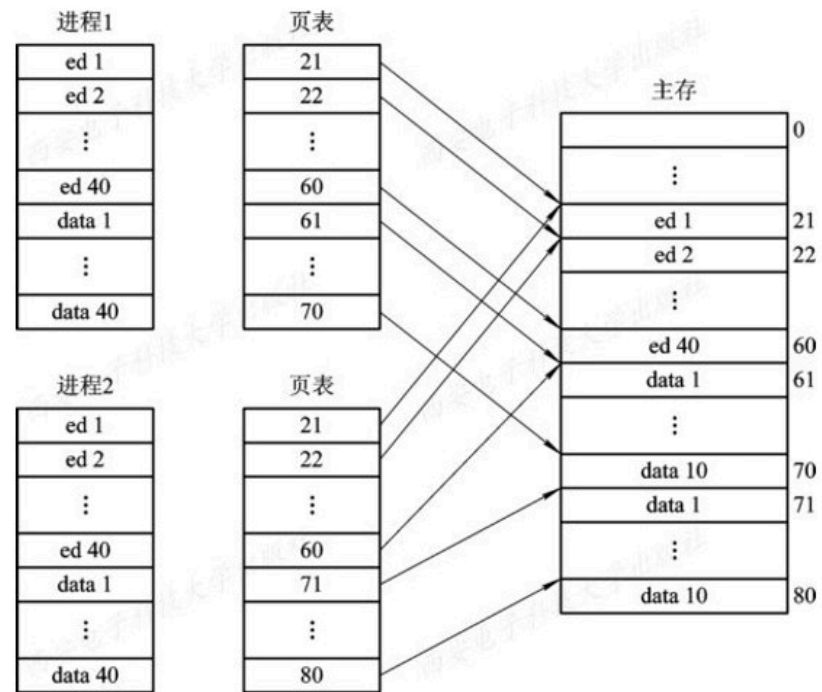
4. 分页和分段的主要区别

- (1) 页是信息的物理单位。
- (2) 页的大小固定且由系统决定。
- (3) 分页的用户程序地址空间是一维的。

4.6.3 信息共享

1. 分页系统对程序和数据的共享

在分页系统中，虽然也能实现对程序和数据的共享，但远不如分段系统来得方便。

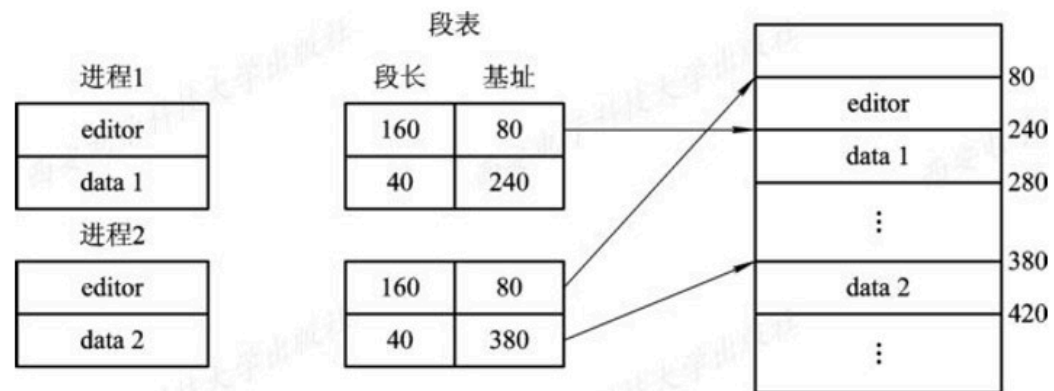


• 图4-21 分页系统中共享editor的示意图

2. 分段系统中程序和数据的共享

在分段系统中，由于是以段为基本单位的，不管该段有多大，我们都只需为该段设置一个段表项，因此使实现共享变得非常容易。

以共享editor为例，此时只需在(每个)进程1和进程2的段表中，为文本编辑程序设置一个段表项，让段表项中的基址(80)指向editor程序在内存的起始地址。图4-22是分段系统中共享editor的示意图。



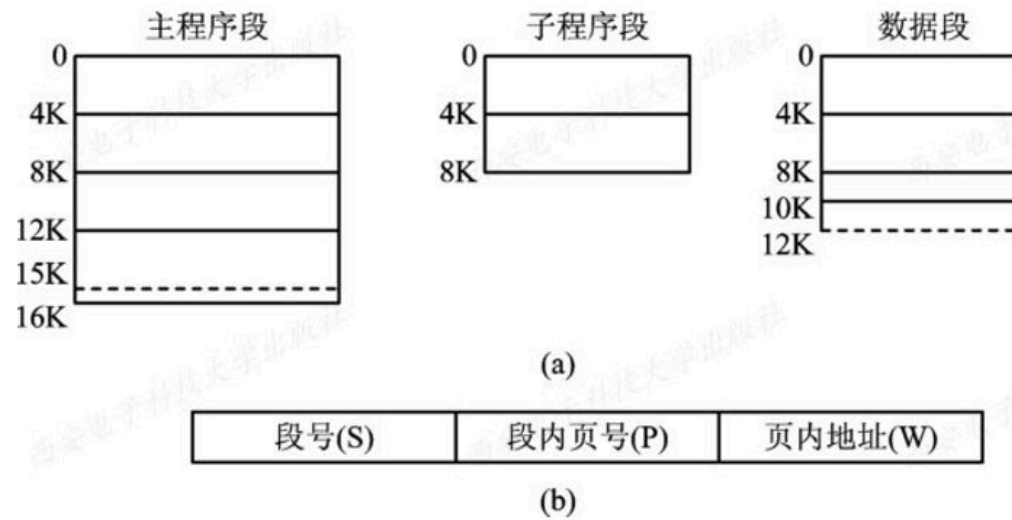
• 图4-22 分段系统中共享editor的示意图

4.6.4 段页式存储管理方式

1. 基本原理

段页式系统的基本原理是分段和分页原理的结合，即先将用户程序分成若干个段，再把每个段分成若干个页，并为每一个段赋予一个段名。

段页式系统中，其地址结构由段号、段内页号及页内地址三部分所组成。

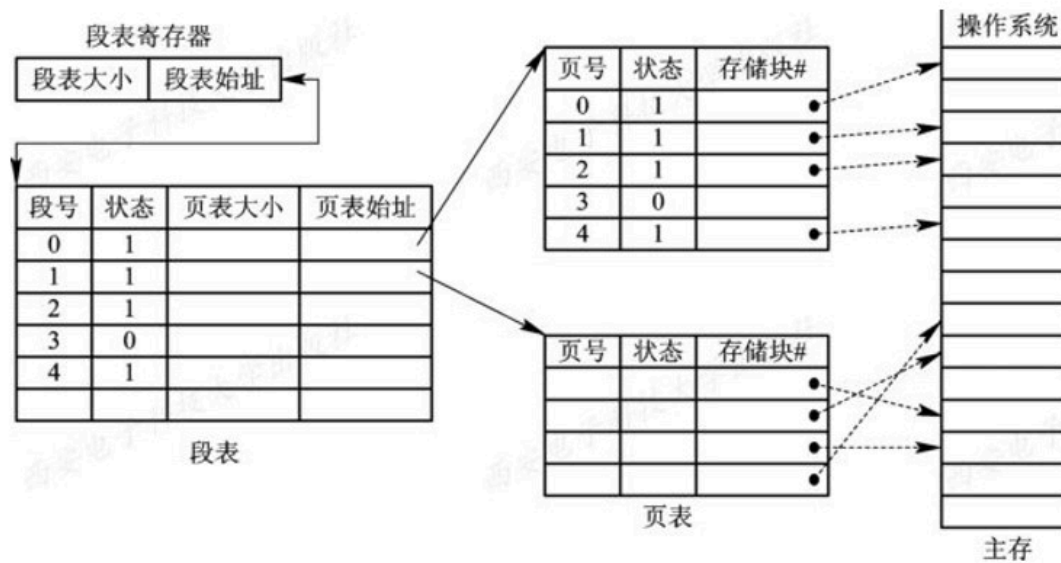


• 图4-23 作业地址空间和地址结构

在段页式系统中，为了实现从逻辑地址到物理地址的变换，系统中需要同时配置段表和页表。

段表的内容与分段系统略有不同，它不再是内存始址和段长，而是页表始址和页表长度。

图4-24示出了利用段表和页表进行从用户地址空间到物理(内存)空间的映射。

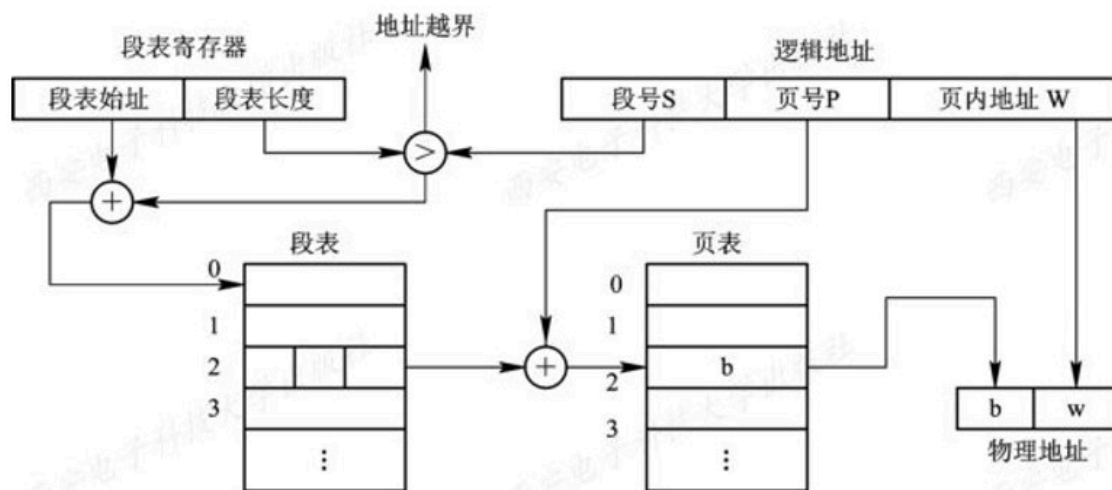


• 图4-24 利用段表和页表实现地址映射

2. 地址变换过程

在段页式系统中，为了便于实现地址变换，须配置一个**段表寄存器**，其中存放段表始址和段长TL。

进行地址变换时，首先利用段号S，将它与段长TL进行比较。若 $S < TL$ ，表示未越界，于是利用段表始址和段号来求出该段所对应的段表项在段表中的位置，从中得到该段的**页表始址**，并利用逻辑地址中的段内页号P来获得对应页的页表项位置，从中读出该页所在的物理块号b，再利用块号b和页内地址来构成物理地址。



• 图4-25 段页式系统中的地址变换机构

主观题 10分

在本课堂上，你想学习那些内容？
你有那些建议？



投票(匿名) 最多可选5项

在线听课评价:

- ☐ A 教师准备充分,讲述条理清晰
- ☐ B 能调动学习的积极性
- ☐ C 教学方式单一、缺乏互动
- ☐ D 学生的收获少, 获得感不强
- ☐ E 内容枯燥、听不懂

